

Document Number: P2835R2

Date: 2024-01-10

Reply to: Gonzalo Brito Gadeschi <gonzalob _at_ nvidia.com>

Authors: Gonzalo Brito Gadeschi

Audience: LEWG

Expose `std::atomic_ref` 's object address

Changelog

- R2: Preparation for mailing list review
 - Update links to compiler explorer.
 - Update API design rationale.
 - Update `__cpp_lib_atomic_ref` macro.
- R1:
 - Add alternative API designs.
- R0: initial revision (Varna)

Introduction

`std::atomic_ref` prevents applications from obtaining the address of the object referenced by `*this` and, therefore, from reasoning about contention on accesses to the object, which is crucial for performance (see “Usecases” section).

Applications that need to reason about contention for performance cannot use `std::atomic_ref` but may be able to use `std::atomic&` or `std::atomic*` instead.

That is not always possible, e.g., if object’s type is outside the application’s control. Then, a `pair<atomic_ref<T>, T*>` may be passed around instead. However, this is not ergonomic, and always having a pointer available slightly increases the hazard of accidentally accessing the object via a raw pointer while an `atomic_ref` object is still live.

This paper proposes to add a `.data()` member function to `std::atomic_ref` instead, which can be used when the application needs to access the underlying object’s address, e.g., to be able to reason about contention.

Tony tables

Before	After
<pre>std::atomic<int>& ref; auto* addr = &ref;</pre>	<pre>std::atomic_ref ref; auto* addr = ref.data();</pre>

Alternatives

Currently, it is not possible to obtain a pointer to the underlying object of an `std::atomic_ref`, and therefore not possible to accidentally access the object concurrently through a raw pointer while the `std::atomic_ref` is still live.

The proposed API introduces a `data` member function that returns a `T const*`. A program that accidentally dereferences this pointer while there are live `std::atomic_ref` referencing the object exhibits undefined behavior.

To make this accidental usage of this API harder, we could:

- Change the return type to `void const*`: `void const* data() const noexcept;`
- Change the return type to `uintptr_t` and the name to `address : uintptr_t address() const noexcept;`

Wording

Add the following to [\[atomics.ref.generic.general\]](#).

```
namespace std {
    template<class T> struct atomic_ref {
        // ...
        T const* data() const noexcept;
        // ...
    };
}
```

Add the following to [\[atomic.ref.ops\]](#):

```
T const* data() const noexcept;
```

◦ _____

◦ _____

*** Returns:** pointer to the object referenced by `*this`.

Update `__cpp_lib_atomic_ref` version macro in [<version> synopsis](#) [\[version.syn\]](#) to the C++ version this feature is introduced in: _____

```
#define __cpp_lib_atomic_ref 201806L // freestanding, also in <atomic>
```

Use cases

The main use case is detecting contention, and using that information to optimize concurrent algorithms.

Discovery Patterns

Some hardware architectures have instructions to “discover” different threads of the same program that are running on the same core and are execution the same “program step”.

In those hardware architectures, these instructions can be used to aggregate atomic operations performed by different threads into a single operation performed by one thread. The pattern looks like this:

```
void unsynchronized_aggregated_faa(atomic<int>& acc, int upd) {  
    // Find all spatially-close threads executing this program step  
    // with same values of "acc" and "upd".  
    auto thread_mask = __discover_threads_with_same(acc, upd);  
    auto thread_count = popcount(thread_mask);  
  
    // These threads elect a leader, which aggregates their updates  
    // and performs a single atomic RMW operation instead of one  
    // per thread:  
    if(__pick_one(thread_mask))  
        acc.fetch_add(thread_count * upd, memory_order_relaxed);  
}
```

On NVIDIA GPUs, this optimization can significantly increase the performance of certain algorithms, like “arrive” operations on barriers. In this example ([godbolt](#)), even with a small number of threads, ~1.25x speed ups are measured.

- _____
- _____
- _____
- _____
- _____